Software Architecture and Design Specification (SAD)

Change Management Software

Project	Change Management Software (CMS)
Version	1.0
Authors	Four-member Software Engineering project team
Date	28 October 2026
Status	Draft for team review

Revision History

Version	Date	Author	Change summary	Approval
1.0	28 October 2026	Four-member Software Engineering project team	Initial project-specific draft	Pending

Approvals

Role	Name	Signature / Email	Date
Project team representative	To be filled by team	Pending	
Course instructor / TA	To be filled by instructor	Pending	

Table of Contents

- 1. Introduction
- 2. Architecture Goals and Constraints
- 3. Stakeholders and Concerns
- 4. Architecture and Components
- 5. Architecture Pattern and Technology Stack
- 6. Data Design
- 7. API / Service Design
- 8. Sequence Flows
- 9. Security Architecture
- 10. Error Handling, Logging and Monitoring
- 11. Traceability, Risks and Open Issues

1. Introduction

1.1 Purpose

This document describes the proposed architecture and design for CMS. It defines component responsibilities, data relationships, the change request workflow model, service boundaries, important flows, security controls, and traceability to the SRS.

1.2 Scope

The design covers change request creation and editing, the status workflow, review and approval, assignment, comments and work notes, resolution, verification and closure, reopening, SLA/overdue tracking, in-app notifications, reports, authentication, authorization, and audit history. Email, chat, source-control and CI/CD integrations are optional future work.

2. Architecture Goals and Constraints

- Correct, consistently enforced workflow transitions and a complete, traceable history are primary goals.
- Keep workflow and assignment rules in one place and testable independently of UI code.
- Use a small, understandable architecture suitable for a four-member student team and sprint delivery.
- Persist data reliably, validate all user input, and prevent concurrent updates from silently overwriting each other.
- Match final diagrams and technology descriptions to the implementation; record deviations before submission.

3. Stakeholders and Concerns

Stakeholder	Primary concern
Requester	Quick CR creation, visibility of progress, ability to confirm or reopen.
Change manager / reviewer	Clear triage queue, fair workload assignment, overdue visibility, reports.
Engineer	Clear list of assigned work, easy status updates, full context on each CR.
Administrator	User access, configuration of components and SLA targets, auditability.
Developers	Modular code, testable workflow rules, clear interfaces.
Evaluator / TA	Traceability from requirements to implementation, tests, commits, and demos.

4. Architecture and Components

4.1 Chosen Pattern

Use a layered modular-monolith architecture: presentation/UI, application routes/controllers, business services, repositories/data-access, and relational database. The workflow rules are implemented as an explicit state machine inside the business layer so every status change goes through one tested component. This keeps deployment and collaboration simple while allowing service-level unit tests. A microservices architecture is not recommended for the initial prototype because it would add operational overhead without a demonstrated need.

4.2 Component Responsibilities

Component	Responsibilities	Key interactions
Presentation layer	Dashboard, CR list and detail pages, forms, history timeline, notifications panel, user feedback.	Calls routes/controllers; renders results.
Authentication / authorization	Sign-in/out, session or token validation, role checks, record-level ownership checks.	Protects routes and service operations.
Change request service	Create, read, edit and search CRs; field validation; ID generation.	Uses CR repository; calls Workflow Engine for status changes; emits audit event.
Workflow engine	Holds the allowed-transition table; validates and applies status changes and their required inputs (reason, resolution summary).	Writes status history; triggers notifications and audit.
Assignment service	Assign/reassign CRs; check the target user is an active engineer.	Uses user and CR repositories; calls Workflow Engine to move CR to Assigned.
Comment / activity service	Store comments and work notes; build the CR activity timeline.	Reads history and comments; triggers notifications.
SLA service	Compute due dates from priority and settings; identify overdue CRs.	Reads CRs and settings; feeds dashboard and reports.
Notification service	Create, list and mark-read in-app notifications.	Called by workflow, assignment and comment services.
Reporting service	Open-CR, workload, ageing and resolution-time reports; dashboard counts.	Queries CR, history and user data.
Audit service	Record important actions and actor/timestamp details.	Called by CR, workflow, assignment and user-management services.
Repository / data-access layer	Encapsulate database queries, transactions and optimistic locking.	Used by business services.
Relational database	Persist users, roles, CRs, history, comments, notifications, settings and audit events.	Accessed only through data-access layer.

4.3 Component Diagram (text specification)

Browser UI → Routes/Controllers → (Authentication & Authorization; Change Request Service; Workflow Engine; Assignment Service; Comment/Activity Service; SLA Service; Notification Service; Reporting Service; Audit Service) → Repositories → Relational Database. Change Request and Assignment services delegate every status change to the Workflow Engine. The Workflow Engine, Assignment Service and Comment Service call Notification Service and Audit Service after successful changes. SLA and Reporting services only read data.

5. Technology Stack and Data Stores

Use the technologies already selected in the repository. The proposed implementation is a Python web application (for example Flask or FastAPI) with service modules, a relational database (SQLite for local development and PostgreSQL or MySQL for a shared environment), and a browser UI. The precise framework and database should be recorded from the actual repository rather than assumed in this draft.

Development tooling: Git/GitHub branches and pull requests, automated checks through GitHub Actions, unit tests for business services, and a README describing environment setup. Production hosting and external integrations are outside the initial scope.

6. Data Design

Entity	Important fields	Relationships / rules
User	id, name, email, password_hash, role_id, active	Role assigned to each user; never store plaintext password; inactive users cannot sign in or be assigned.
Role	id, name, permissions	Controls protected operations (Admin, Reviewer, Engineer, Requester, Viewer).
Component	id, name, active	One component can have many CRs.
ChangeRequest	id, cr_number, title, description, type, priority, component_id, status, requester_id, assignee_id, due_date, resolution_summary, created_at, updated_at, closed_at, version	Belongs to one requester and at most one assignee; version supports optimistic locking.
StatusHistory	id, cr_id, from_status, to_status, reason, user_id, timestamp	Append-only; one row for every status change, including reopen.
AssignmentHistory	id, cr_id, from_user_id, to_user_id, assigned_by, timestamp	Append-only; one row for every assignment or reassignment.
Comment	id, cr_id, author_id, body, is_work_note, created_at	Belongs to one CR and one author.
Notification	id, user_id, cr_id, type, message, read, created_at	Created for assignee/requester on relevant events.
SystemSetting / SlaPolicy	priority, target_days, updated_by, updated_at	Stores SLA targets and defaults.
AuditEvent	id, user_id, action, entity_type, entity_id, timestamp, summary	No passwords or secrets in event payloads.

6.1 Workflow State Model

From status	Allowed next status	Who / required input
Submitted	Under Review	Reviewer
Under Review	Approved or Rejected	Reviewer; rejection reason required
Approved	Assigned	Reviewer; active engineer required
Assigned	In Progress; Assigned (reassign)	Assigned engineer starts work; reviewer reassigns
In Progress	Resolved; Assigned (reassign)	Assigned engineer; resolution summary required
Resolved	Closed or Reopened	Requester or reviewer; reopen reason required
Reopened	In Progress; Assigned (reassign)	Assigned engineer or reviewer
Rejected / Closed	None (terminal)	Changes only through an administrator correction, which is audited

6.2 Data Integrity Rules

- Use database constraints (foreign keys, NOT NULL, allowed values) where supported and repeat important validation in the service layer.
- Change a CR's status only through the Workflow Engine, and write the status change and its history row in one database transaction.
- Use a version column on ChangeRequest; an update whose version does not match the stored value is rejected as a conflict.
- Keep StatusHistory, AssignmentHistory and AuditEvent append-only.
- Store timestamps in UTC and convert to the configured local time zone for display and due-date checks.

7. API / Service Design

The exact URL routing may follow the repository's framework. The following is a proposed service/API contract; adapt it to the implementation and document any differences.

Operation	Method / route (example)	Input	Successful result / errors

List/search CRs	GET /api/change-requests	query, status, priority, assignee, component, date range	CR list; 400 for invalid filter.
Create CR	POST /api/change-requests	title, description, type, priority, component_id	Created CR with status Submitted; 400 validation error; 401 not signed in.
Get CR detail	GET /api/change-requests/{id}	id	CR with comments and history; 403 no access; 404 not found.
Edit CR	PATCH /api/change-requests/{id}	changed fields, version	Updated CR; 400 invalid; 403 not permitted; 409 version conflict or CR closed.
Change status	POST /api/change-requests/{id}/transitions	to_status, reason / resolution_summary, version	Updated CR and history row; 400 missing input; 403 wrong role; 409 transition not allowed.
Assign / reassign	POST /api/change-requests/{id}/assignment	assignee_id, version	CR assigned; 400 assignee not an active engineer; 403 not reviewer.
Add comment	POST /api/change-requests/{id}/comments	body, is_work_note	Comment created; 400 empty body; 403 no access.
Notifications	GET /api/notifications; POST /api/notifications/{id}/read	unread filter	Notification list; marked as read.
Dashboard / reports	GET /api/dashboard; GET /api/reports/{type}	date range and filters	Counts or report data; 400 invalid range; 403 if role is unauthorized.

8. Sequence Flows

8.1 Create Change Request

- Requester opens the new-request form and submits title, description, type, priority and component.
- UI sends the request to the route/controller.
- Controller checks authentication, then calls Change Request Service.
- Service validates required fields and allowed values, generates the CR number, and sets status Submitted and due date from the SLA policy.
- Service writes the CR and its first StatusHistory row in one database transaction.
- Audit Service records the creation; Notification Service notifies reviewers; controller returns the created CR.
- If validation or persistence fails, the transaction is rolled back and a safe error is returned.

8.2 Approve, Assign and Resolve

- Reviewer opens a Submitted CR, moves it to Under Review, and approves it through the transitions endpoint.
- Reviewer selects an engineer; Assignment Service checks the user is an active engineer and calls the Workflow Engine to move the CR to Assigned.
- The CR update, StatusHistory and AssignmentHistory rows are written in one transaction; the engineer receives a notification.
- Engineer starts work (In Progress), adds work notes, and submits a resolution summary to move the CR to Resolved.
- Workflow Engine rejects the resolve step if the summary is empty or the caller is not the assignee.

8.3 Verify and Close or Reopen

- Requester or reviewer opens a Resolved CR and reviews the resolution summary.
- To close, they submit the Closed transition; the closure date is set and the CR leaves the open and overdue lists.
- To reopen, they submit the Reopened transition with a reason; the assignee is notified and the CR returns to their queue.
- Audit and history records are written for either outcome; dashboard and reports reflect the change.

9. Security Architecture

Threat / concern	Control
Unauthorized access	Authentication plus server-side role checks for every protected operation.
Acting on someone else's CR	Record-level checks: only the assignee can resolve; only the requester or a reviewer can close or reopen.
Tampering with workflow or history	All status changes go through the Workflow Engine; history and audit tables are append-only.
Credential disclosure	Salted password hashing, HTTPS in deployed environments, secret-free logs.
Injection / cross-site scripting	Parameterized database access, strict input validation, and output escaping of user-entered text.
Privilege escalation	Deny-by-default permissions and tests for direct requests by unauthorized roles.
Lost updates	Optimistic locking with a version field; conflicts return 409.
Accidental data loss	Database backup/export procedure and documented recovery steps appropriate to the prototype.

Threat-modeling prompts: spoofing (fake user/session), tampering (changing status or history outside the workflow), repudiation (denial of an approval, rejection or closure), information disclosure (credentials or CR content exposed to the wrong user), denial of service (invalid or excessive requests), and elevation of privilege (engineer or requester performing reviewer/admin operations).

10. Error Handling, Logging and Monitoring

- Return consistent validation, permission, transition and conflict errors; do not show stack traces to end users.
- Log unexpected failures with a correlation/reference ID where feasible; never log passwords or secrets.
- Record business audit events separately from diagnostic logs.
- Monitor application startup failures, failed operations, rejected transitions, version conflicts, and test results.
- Provide an understandable empty state when no CRs, notifications or report rows match the selected filter.

11. Traceability, Risks and Open Issues

SRS requirement	Design component
CMS-F-001 / CMS-SR-001	Authentication / authorization
CMS-F-003 to CMS-F-006	Change Request Service and CR Repository
CMS-F-007, F-008, F-011, F-012	Workflow Engine and StatusHistory
CMS-F-009 / CMS-SR-005	Assignment Service, AssignmentHistory, record-level authorization
CMS-F-010	Comment / Activity Service
CMS-F-013	StatusHistory, AssignmentHistory, activity timeline
CMS-F-014	SLA Service
CMS-F-015	Notification Service
CMS-F-016 to CMS-F-018	Reporting Service
CMS-F-019 / CMS-SR-006	Audit Service
CMS-NF-001	Indexes on status, assignee, priority and dates; performance test suite
CMS-NF-005	Layered modular architecture and service unit tests
CMS-NF-007	Version field and optimistic locking in the repository layer

11.1 Risks and Mitigations

Risk	Mitigation
Workflow rules scattered across UI and routes	Keep one transition table in the Workflow Engine; unit-test every allowed and disallowed transition.
History out of sync with current status	Write status and history in one transaction; add a consistency test.

Due-date and time-zone errors	Store UTC, define local-time rules, and test due today, due tomorrow, and past-due cases.
Merge conflicts across four contributors	Assign clear module ownership, use feature branches and pull requests, integrate frequently.
Scope expansion (email, Git integration, attachments)	Prioritize the core create/assign/resolve/close flow; label integrations as future work.
Requirements differ from delivered product	Maintain RTM and document deviations for final demonstration.

11.2 Open Issues

- Confirm the final web framework and database from the repository.
- Confirm whether file attachments on CRs are required in the first release.
- Confirm whether notifications are in-app only or also require email.
- Confirm whether a CR can be reassigned while In Progress without returning to Approved.
- Populate the final component/class/sequence diagrams using the actual module names before submission.